

Práctica 5

1. ¿Cuál es la función de la capa de transporte?

"Mientras que la capa de red proporciona delivery end-to-end de paquetes utilizando datagramas o circuitos virtuales, la **capa de transporte** se sitúa sobre la capa de red para brindar transporte de datos desde un proceso en un host emisor a un proceso en un host receptor, con un grado de fiabilidad bueno que es independiente de las redes físicas que se usan como medio. Provee abstracciones que las aplicaciones utilizan para usar la red." [Tenenbaum, p. 495]

"Gracias a la capa de transporte, los programadores de aplicaciones pueden escribir códigos acorde a un conjunto de primitivos básicos, y hacer que esto programsa corran en una gran variedad de redes, sin tener que preocuparse con los detalles sobre como lidiar con interfaces de redes y grados de fiabilidad diferentes. [...] en el mundo real cumple el rol de aislar a las capas superiores de la tecnología, el diseño y las imperfecciones de la red." [Tenenbaum, p. 497]

2. Describa la estructura del segmento TCP y UDP

Segmento UDP

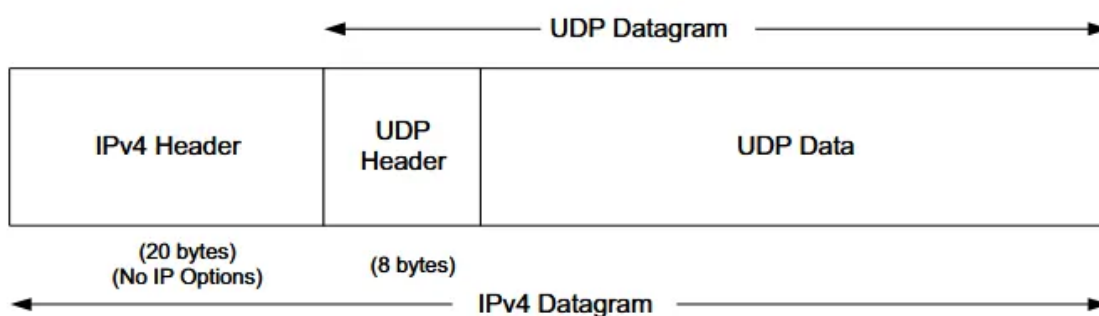


Figura 10.1, Stevens. *Paquete UDP (IPv4) y su contenido.*

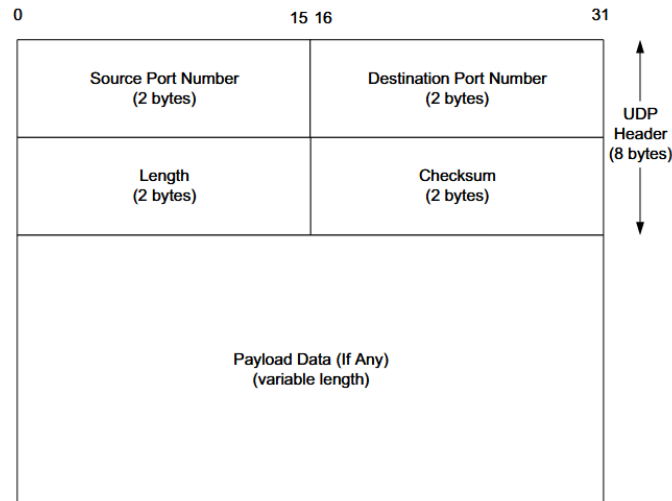


Figura 10.2, Stevens. *Encabezado del UDP*

Un **segmento UDP** (paquete/datagrama?) está conformado por 4 campos principales y uno reservado para la data recibida:

- **Puerto emisor** → Proceso emisor en el host fuente (?)
- **Puerto receptor** → Proceso receptor en el host destino (?)
- **Longitud** → Longitud del encabezado del UDP y de su data en bytes
- **Checksum** → Función de redundancia que tiene como propósito detectar cambios accidentales en una secuencia de datos para proteger la integridad de estos. *Se calcula al inicio y al final de la transmisión.*

Y luego, para la parte de **datos...**

"Los datos de la aplicación ocupan el campo de datos del segmento UDP. Por ejemplo, para DNS, el campo de datos contiene un mensaje de consulta o un mensaje de respuesta. En el caso de una aplicación de flujo de audio, las muestras del audio llenarán el campo de datos" [Kurose, p. 167]

Segmento TCP

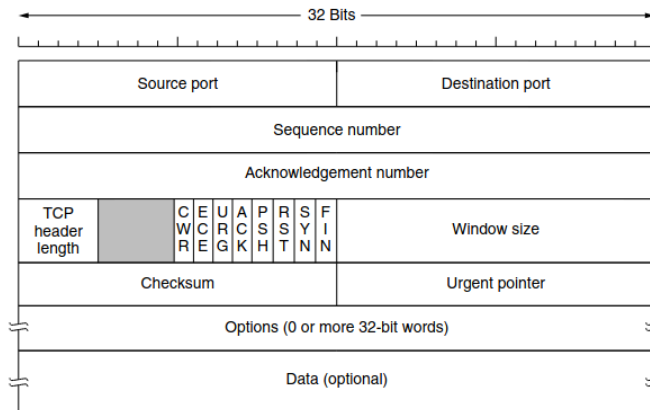


Figure 6.36, Tenenbaum. Encabezado TCP

Un segmento TCP cuenta con 12 campos en su encabezado.

- **Puerto emisor**
- **Puerto receptor**
- **Número de secuencia** → Es el número del byte del flujo que se envía
 "El uso que hace TCP de los números de secuencia refleja este punto de vista, en el sentido de que los números de secuencia hacen referencia al flujo de bytes transmitido y no a la serie de segmentos transmitidos. **El número de secuencia de un segmento es por tanto el número del primer byte del segmento dentro del flujo de bytes**" [Kurose, p. 195]

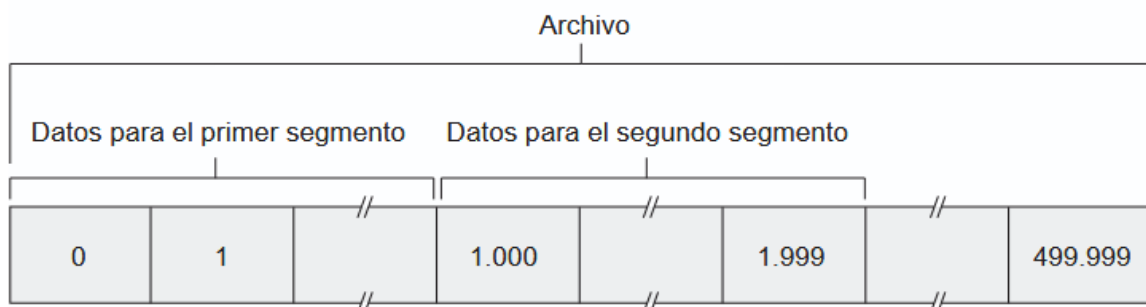


Figura 3.30 ♦ División de los datos del archivos en segmentos TCP.

- **Número de reconocimiento** → Es el byte siguiente al byte de todo el flujo que recibió hasta el momento el receptor, es "el número de secuencia del siguiente byte que el host A está esperando del host B"
- **Longitud del encabezado TCP**

- Un campo de 4 bits que no se utiliza (**ver**)
- **Un campo de flags de 1-bit** → Incluye el de congestión, el urgent, etc...
"El bit ACK se utiliza para indicar que el valor transportado en el campo de reconocimiento es válido; es decir, el segmento contiene un reconocimiento para un segmento que ha sido recibido correctamente. Los bits RST, SYN y FIN se utilizan para el establecimiento y cierre de conexiones. Los bits CWR y ECE se emplean en la notificación de congestión explícita. La activación del bit PSH indica que el receptor deberá pasar los datos a la capa superior de forma inmediata. Por último, el bit URG se utiliza para indicar que hay datos en este segmento que la entidad de la capa superior del lado emisor ha marcado como "urgentes"." [Kurose, p. 194]
- **Tamaño de la ventana** → "Indica cuantos bytes (paquetes) se pueden enviar desde el byte reconocido" [Tenenbaum, p. 559]
- **Checksum** → Realiza un checksum del encabezado, los datos, y un pseudoheader para TCP
- **Urgent Pointer** → Contiene datos urgentes que se tienen que leer primero si se establece
- **Opciones** → "Se utiliza cuando un emisor y un receptor negocian el tamaño máximo de segmento (MSS) o como un factor de escala de la ventana en las redes de alta velocidad. También se define una opción de marca temporal. Consulte los documentos RFC 854 y RFC 1323 para conocer detalles adicionales" [Kurose, p. 194]
- **Datos (opcional)**

3. ¿Cuál es el objetivo del uso de puertos en el modelo TCP/IP?

- "En la capa de transporte, la mayoría de protocolos usan números de puertos para demultiplexar para la aplicación receptora apropiada." [Stevens, p. 17]
- "Cada dirección IP tiene 65,536 números de puertos asociados para cada protocolo de transporte que utilice números de puertos."
- "Para aplicaciones cliente/servidor, un servidor primero "se vincula" a un número de puerto, y luego, uno o mas clientes establecen conexiones a ese puerto utilizando un protocolo de transporte en particular, en una máquina

particular. En este sentido, los números de puertos (o los puertos simplemente) cumplen la función de "extensiones de números de teléfono."

4. Compare TCP y UDP en cuanto a:

- **Confiabilidad** → A diferencia de UDP, TCP si provee mecanismos para corregir errores y prevenir que el tráfico alto de paquetes impacte negativamente sobre otros usuarios.
- **Multiplexación** → **UDP realiza esta función**, agregando los campos de puertos (y el checksum?) al paquete, en la parte del encabezado. Para que se distinga qué protocolo de transporte se utilizó una vez que se está procesando el datagrama IP, se le coloca a este en su campo *Protocolo* (Protocol) el número 17,
- **Orientado a la conexión** → UDP no es un protocolo orientado a la conexión, TCP si
"Observe que con UDP no tiene lugar una fase de establecimiento de la conexión entre las entidades de la capa de transporte emisora y receptora previa al envío del segmento. Por esto, se dice que UDP es un protocolo sin conexión" [Kurose, p. 164]
- **Controles de congestión** → Mientras que TCP ofrece (vease TCP Tahoe, Reno, ...), UDP no
"TCP dispone de un mecanismo de control de congestión que regula el flujo del emisor TCP de la capa de transporte cuando uno o más de los enlaces existentes entre los hosts de origen y de destino están excesivamente congestionados" [Ídem]
- **Utilización de puertos** → Los puertos se utilizan de la misma forma ?

5. La PDU de la capa de transporte es el segmento. Sin embargo, en algunos contextos suele utilizarse el término datagrama. Indique cuando

"No obstante, tenemos que decir que, en textos dedicados a Internet, como por ejemplo los RFC, también se emplea el término segmento para hacer referencia a los paquetes de la capa de transporte en el caso de TCP, pero a menudo a los paquetes de UDP se les denomina datagrama" [Kurose, p. 156]

En general, se utiliza el término datagrama para referirse a los paquetes UDP, y segmentos para referirse a los paquetes TCP.

6. Describa el saludo de tres vías de TCP. ¿UDP tiene esta característica?

[Kurose, p. 209 - 210]

1. "TCP del lado del cliente envía un segmento TCP especial al TCP del lado servidor. Este segmento especial no contiene datos de la capa de aplicación. Pero uno de los bits indicadores de la cabecera del segmento, el bit SYN, se pone a 1."
2. "Una vez que el datagrama IP que contiene el segmento SYN TCP llega al host servidor (¡suponiendo que llega!), el servidor extrae dicho segmento SYN del datagrama, asigna los buffers y variables TCP a la conexión y envía un segmento de conexión concedida al cliente TCP. "
3. "Al recibir el segmento SYNACK, el cliente también asigna buffers y variables a la conexión. El host cliente envía entonces al servidor otro segmento; este último segmento confirma el segmento de conexión concedida del servidor. El bit SYN se pone a cero, ya que la conexión está establecida. Esta tercera etapa del proceso de acuerdo en tres fases puede transportar datos del cliente al servidor dentro de la carga útil del segmento."

No, UDP no tiene la tarea de verificar si el paquete llegó al host destino u no.

7. Investigue qué es el ISN (Initial Sequence Number). Relaciónelo con el saludo de tres vías.

El ISN es un campo de los segmentos TCP, y que utilizan los protocolos de transferencia de datos fiables en general, que ayuda tanto a la identificación como a reconocimiento de estos durante una comunicación. Puntualmente, su propósito es el de obfuscar el comienzo de la comunicación (dps veo, numero no adivina para cual comenzar de cda lado):

1. El emisor podría recibir un segmento de confirmación corrupto (¿quería decir ACK o NAK?)
2. El último segmento podría haberse perdido, y el receptor no lo recibió

El ISN y el sequence number es el mismo campo pero se usan para cosas diferentes

"Una solución sencilla [al problema de corroborar que se haya recibido el segmento que había que enviar, sin que sufriera alteraciones o se perdiera] consiste en añadir un nuevo campo al paquete de datos, y hacer que el emisor numere sus paquetes de datos colocando un **número de secuencia** en este campo. Entonces bastará con que el receptor compruebe ese *número de secuencia* para determinar si el paquete recibido es o no una retransmisión."

[Kurose, p. 174]

Para generar ese numero aleatorio se le consulta el SO

8. Investigue qué es el MSS. ¿Cuándo y cómo se negocia?

El MSS es el tamaño máximo del segmento (maximum segment size) que se transmitirá en un canal de comunicación establecido de TCP. Este número suele ser negociado durante la etapa del saludo de tres vías (Triple Handshake), y se establece escogiendo el tamaño de segmento menor de los comunicados por el emisor o el receptor en los paquetes del saludo.

"Observe que el MSS es la cantidad máxima de datos de la capa de aplicación en el segmento, no el tamaño máximo del segmento TCP incluyendo las cabeceras"

[Kurose, p. 193]

9. Utilice el comando ss (reemplazo de netstat) para obtener la siguiente información de su PC:

- a. ss -t
- b. ss -u
- c. ss -lt
- d. ss -lu
- e. .
- f. netstat -au / netstat -at

10. ¿Qué sucede si llega un segmento TCP con el flag SYN activo a un host que no tiene ningún proceso esperando en el puerto

destino de dicho segmento (es decir, el puerto destino no está en estado LISTEN)?

- Depende de si está en estado LISTEN o no, lo recibe (con flag SA) o no (con flag RA)

11. ¿Qué sucede si llega un datagrama UDP a un host que no tiene ningún proceso esperando en el puerto destino de dicho datagrama (es decir, que dicho puerto no está en estado LISTEN)?

- En el primer caso, los paquetes se envían pero ninguno llega dice (pérdida del 100% de paquetes), mientras que en el segundo caso los paquetes no se envían porque dice que "el puerto es inalcanzable"
- Como el puerto 5353 está activo, está listo para recibir paquetes, pero no responde a la conexión que quiero establecer porque el protocolo UDP no establece conexiones, si no que recibe paquetes UDP. Pero el puerto 40 ni siquiera está prendido.

12. Investigue los distintos tipos de estado que puede tener una conexión TCP.

- **LISTEN** → Representa la espera por una petición de conexión desde cualquier TCP y puerto remoto
- **SYN-SENT** → Representa la espera por una petición de conexión positiva luego de haber enviado una petición de conexión
- **SYN-RECEIVED** → Representa la espera por el reconocimiento de las peticiones de conexión confirmadoras luego de que ambos hayan recibido y enviado una petición de conexión
- **ESTABLISHED** → Representa una conexión abierta, los datos recibidos pueden ser enviados al usuario. Es el estado normal para la fase de transferencia de datos de la conexión
- **FIN-WAIT-1** → Representa la espera por una petición de termino de conexión desde el TCP remoto, o un reconocimiento de una petición de termino de conexión enviado con anterioridad

- **FIN-WAIT-2** → Representa la espera por una petición de termino de conexión por parte del TCP remoto
- **CLOSE-WAIT** → Representa la espera por una petición de termino de conexión por parte del usuario local
- **CLOSING** → Representa la espera por el reconocimiento de una petición de termino de conexión por parte del TCP remoto
- **LAST-ACK** → Representa la espera por el reconocimiento de una petición de termino de conexión anteriormente enviada al TCP remoto (que incluye un reconocimiento de su petición de termino de conexión)
- **TIME-WAIT** → Representa la espera de que pase tiempo suficiente para asegurarse de que el TCP remoto recibió el "reconocimiento" de su petición de termino de conexión
- **CLOSED** → Representa la ausencia de un estado de conexión

13. Dada la siguiente salida del comando ss, responda:

a. **¿Cuántas conexiones hay establecidas?**

Hay 8 conexiones establecidas. Por mas que se observen 9 procesos TCP en estado ESTABLISHED, 2 de esos 9 procesos pertenecen a la misma conexión (el Local Address:Port es el mismo que el Peer Address:Port de la otra conexión) y entonces no se tiene en cuenta

b. **¿Cuántos puertos hay abiertos a la espera de posibles nuevas conexiones?**

Hay tres puertos locales abiertos a la espera de posibles conexiones: el puerto 22, el puerto 80, y el puerto 25. Luego también hay dos puertos abiertos UDP, pero el UDP es un protocolo sin conexión, o que no está orientado a la conexión, por lo que esos dos puertos no se utilizan para establecer conexiones.

c. **El cliente y el servidor de las comunicaciones HTTPS (puerto 443), ¿residen en la misma máquina?**

No, ya que no se observa que se utilice ningún puerto 443 de forma local.

d. **El cliente y el servidor de la comunicación SSH (puerto 22), ¿residen en la misma máquina?**

Si, ya que podemos ver las dos direcciones de la conexión desde el lado local y el remoto en la misma máquina

- e. **Liste los nombres de todos los procesos asociados con cada comunicación. Indique para cada uno si se trata de un proceso cliente o uno servidor.**

Son las ESTABLISHED, menos una porque hay que pertenecen a la misma conexión

- f. **¿Cuáles conexiones tuvieron el cierre iniciado por el host local y cuáles por el remoto?**

La que tiene un estado CLOSE-WAIT tuvo el cierre iniciado por el host remoto. La que tiene un estado TIME-WAIT tuvo el cierre iniciado por el host local.

- g. **¿Cuántas conexiones están aún pendientes por establecerse?**

Si solo contamos los puertos que comenzaron a establecer la comunicación, tenemos 1, que es el estado con SYN-SENT. Si contamos además los puertos locales TCP que están escuchando (en estado LISTEN), son 4.

14. Dadas las salidas de los siguientes comandos ejecutados en el cliente y el servidor, responder:

- a. **¿Qué segmentos llegaron y cuales están perdidos en la red?**

Como el puerto del servidor está con el estado SYN-RECEIVED, podemos deducir de que el puerto anda esperando la última de las tres peticiones del Handshake para establecer la conexión, por lo que de cada lado se envió y se obtuvo un paquete. Pero al parecer el paquete, confirmador de la comunicación entre ambas partes, de parte del cliente se perdió.

- b. **¿A qué protocolo de capa de aplicación y de transporte se está intentando conectar el cliente?**

Se está intentando conectar al protocolo POP3

- c. **¿Qué flags tendría seteado el segmento perdido?**

El segmento perdido tendría los siguiente flags y con los siguientes valores:
SYN: 1 / ACK: 1

15. Use CORE para armar una topología como la siguiente, sobre la cual deberá realizar:

Preguntar y hacer mañana en clase

- `ncat -l localhost 8001 -k`
- `ncat -C 10.0.0.20 8001 (-u)`

En base a lo observado en el ítem anterior, ¿es posible iniciar más de una conexión desde el cliente al servidor en el mismo puerto destino? ¿Por qué? ¿Cómo se garantiza que los datos de una conexión no se mezclarán con los de la otra?

Para el cliente es transparente, el SO abre cada conexión con un puerto origen distinto no privilegiado y el puerto destino pedido. Cada tupla dirección puerto origen destino es la que distingue a la conexión.

"Cada encabezado TCP contiene el número de puerto fuente y destino. Estos dos valores, junto a las direcciones IP fuente y destino, identifican de manera única a cada conexión. [...] Es un par de sockets o endpoints (la 4-tupla que conste en la dir. IP del cliente, su número de puerto, la dir. IP del server y su número de puerto) la que identifica únicamente a cada conexión TCP." [[Stevens](#), p. 588]